

Lecture 4: Sets, Functions, and the Asymptotic Cost of Merge Sort

Shuolin Ren

sr2696@yale.edu

BIS 527: Introduction to Health Data Science

Yale School of Public Health

Lecturer: Harsh Parikh • Fall 2026

TL;DR

Sets, quantifiers, and functions provide precise descriptions of inputs and algorithmic costs. This chapter explains membership, counting, mappings, and asymptotic upper, lower, and tight bounds. For merge sort on $n = 2^k$ records, the worst-case comparison count satisfies $T(n) = 2T(n/2) + n - 1$ with $T(1) = 0$. Summing the costs across recursion levels gives $T(n) = n \log_2 n - n + 1$ and growth proportional to $n \log n$.

AI use declaration. Claude Opus 5 was used solely for language polishing and LaTeX layout formatting. I produced all original notes and proofs, and personally verified every detail line by line. Nothing remains unverified.

1 Motivation

Sorting patient measurements makes medians and percentiles easy to compute from their ordered values. The number of comparisons needed to sort a list depends on the procedure and the input size. Bubble sort compares adjacent values and swaps those that are out of order. With each pass shortened by one position, its worst-case comparison count on n records is $n(n - 1)/2$. Merge sort divides the list into halves and merges sorted results. Its comparison count grows more slowly as n increases.

2 Setup and Notation

We assume basic arithmetic with exponents and logarithms. The symbol $:=$ denotes a definition, and $\log_2$ is the base-two logarithm. A list A of length n has entries $A[0], \dots, A[n-1]$. Angle brackets denote lists and braces denote sets. Lists preserve order and repeated entries, whereas sets do not. Thus $|\langle 1, 1, 2 \rangle| = 3$ but $|\{1, 1, 2\}| = 2$. Here $|A|$ denotes list length or set cardinality, according to the object being described.

3 Sets and Quantifiers

Elements and membership. A set is a collection of distinct objects called elements. The statement $2 \in \mathbb{N}$ means that 2 belongs to the natural numbers. The notation $x \notin A$ means that x does not belong to A . Set-builder notation specifies elements by a membership criterion: $\{x \in U : P(x)\}$ contains the elements x of a specified set U for which the property $P(x)$ holds. The colon means “such that”. For example, the primes are $\{x \in \mathbb{N} : x \text{ has exactly two distinct positive divisors}\}$.

Number systems and set difference. We include zero among the natural numbers and use the following conventions:

$$\begin{aligned}\mathbb{N} &:= \{0, 1, 2, 3, \dots\}, & \mathbb{Z} &:= \{\dots, -2, -1, 0, 1, 2, \dots\}, \\ \mathbb{Q} &:= \{p/q : p \in \mathbb{Z}, q \in \mathbb{N} \setminus \{0\}\}, & \mathbb{R} &:= \text{the real numbers}.\end{aligned}$$

The difference $A \setminus B := \{x \in A : x \notin B\}$ removes from A all elements belonging to B . If $A = \{1, 2, 3\}$ and $B = \{3, 4\}$, then $A \setminus B = \{1, 2\}$. In particular, denominators in $\mathbb{Q}$ cannot be zero. We write $A \subseteq B$ when every element of A belongs to B . Proper inclusion $A \subsetneq B$ additionally requires $A \neq B$. Equivalently, $B \supsetneq A$. The classroom notation $\subset$ is used here as $\subsetneq$ to distinguish proper inclusion explicitly: $\mathbb{N} \subsetneq \mathbb{Z} \subsetneq \mathbb{Q} \subsetneq \mathbb{R}$. Figure 1 redraws the nested-set picture.

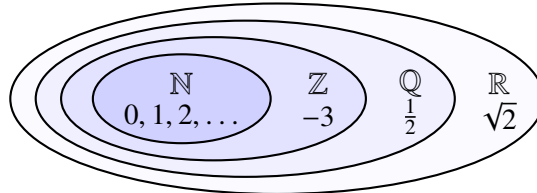


Figure 1: Nested number systems. The examples -3 , $1/2$, and $\sqrt{2}$ witness the successive proper inclusions.

Quantifiers and their order. The symbol $\forall$ means “for every”, and $\exists$ means “there exists”. The abbreviation “s.t.” means “such that”. For example, $\exists x \in \mathbb{N}$ s.t. $2 \mid x$ is true because $x = 4$ is a witness. Here $2 \mid x$ means that x is divisible by 2. Let S be the finite set of BIS 527 students.

Assume each student has exactly one favourite number in $\mathbb{N}$. Write $F(s, x)$ for “ x is student s ’s favourite number”. The classroom statements then have different truth values:

$$\begin{aligned}\forall s \in S \exists x \in \mathbb{N} : F(s, x) & \text{ true,} \\ \forall x \in \mathbb{N} \exists s \in S : F(s, x) & \text{ false.}\end{aligned}$$

At most $|S|$ numbers can be selected, so the choices cannot cover all of $\mathbb{N}$. In the first statement, x may depend on s . The stronger claim $\exists x \in \mathbb{N} \forall s \in S : F(s, x)$ instead requires one number to be every student’s favourite.

4 Functions, Counting, and Input Size

Mappings. Figure 2 pairs numbers with fruits. Its solid arrows assign exactly one fruit to each number. A function $f : A \rightarrow B$ makes exactly one assignment $f(a) \in B$ for every $a \in A$. The sets A and B are its domain and codomain. The value $f(a)$ is the image of a , and a is a preimage of $f(a)$. The image set is $f(A) := \{f(a) : a \in A\}$. Different inputs may have the same image. In the figure, the preimages of banana are 3 and 4. If the dashed arrow is included, 1 has two images, so the relation is not a function.

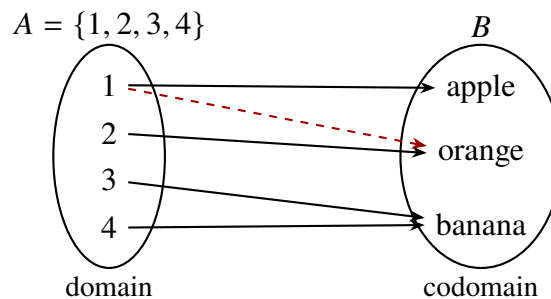


Figure 2: The classroom arrows, with $1 \rightarrow \text{orange}$ shown dashed. The solid arrows alone define a function. All five arrows do not.

In $f(x) = ax + b$, x is the variable and a, b are fixed parameters. For $a = 5, b = 3$, the rule is $f(x) = 5x + 3$. The declarations $f : \mathbb{R} \rightarrow \mathbb{R}$ and $f : \mathbb{N} \rightarrow \mathbb{R}$ specify different domains even with the same rule.

Counting by distinct labels. For $A = \{1, 2, 3\}$ and $B = \{\text{apple}, \text{orange}, \text{banana}\}$, pairing each number with a different fruit gives $|A| = |B| = 3$. Thus the classroom claim $|A| < |B|$ is false, while $|A| \geq |B|$ is true. Giving each fruit a “unique counter” requires a function $c : B \rightarrow A$ that assigns distinct labels to distinct fruits. Such a function is called injective. For finite sets, this is possible only if $|A| \geq |B|$. A statement of the form $\forall b \in B \exists a \in A$ alone does not prevent reuse of a label.

Counting the rationals. For arbitrary sets, equal cardinality means that a bijection exists: a function that pairs every element of one set with exactly one element of the other, with no repetitions or omissions. In fact, $|\mathbb{Q}| = |\mathbb{N}|$. To construct a listing, arrange positive fractions p/q in lowest terms by increasing $p + q$, breaking ties by increasing p . If this list is $r_1, r_2, \dots$, then $0, r_1, -r_1, r_2, -r_2, \dots$ lists every rational exactly once. Its positions provide distinct natural-number counters. Thus proper inclusion does not force unequal sizes for infinite sets (Halmos, 1960).

Slices and costs. For $0 \leq i \leq j \leq n$, the classroom slice notation is

$$A[i:j] := \langle A[i], \dots, A[j-1] \rangle, \quad A[:j] := A[0:j], \quad A[i:] := A[i:n].$$

The slice is empty when $i = j$. Its right endpoint is excluded. For a list, $n = |A|$ counts records, including repeated values. A cost function $T : \mathbb{N} \rightarrow \mathbb{R}$ associates a cost with input size. We use $T(n)$ specifically for the maximum number of comparisons between input values over all lists of length n . This worst-case comparison count excludes copying and loop tests. It represents operation growth, not elapsed seconds.

5 Asymptotic Bounds

The inequality $3n + 2 \leq 5n$ for all integers $n \geq 1$ bounds a cost by a constant multiple of n . Asymptotic notation expresses such comparisons beyond a fixed threshold (Cormen et al., 2001, Section 3.1). Let f, g be positive functions for all sufficiently large integer inputs, and let T be non-negative there. The formal definitions are

$$\begin{aligned} T(n) = O(f(n)) &\iff \exists c > 0 \exists n_0 \forall n \geq n_0 : 0 \leq T(n) \leq cf(n), \\ T(n) = \Omega(g(n)) &\iff \exists d > 0 \exists n_0 \forall n \geq n_0 : 0 \leq dg(n) \leq T(n). \end{aligned}$$

Here n_0 is a positive integer threshold, and c, d are constants independent of n . The notations give upper and lower bounds, respectively (Knuth, 1976). They denote membership in sets of functions, despite the conventional equals sign. A tight bound $T(n) = \Theta(f(n))$ requires both bounds with the same f :

$$\exists c, d > 0 \exists n_0 \forall n \geq n_0 : \quad 0 \leq df(n) \leq T(n) \leq cf(n).$$

If the two bounds initially use different thresholds, take their maximum. Bounds involving unrelated f and g need not be tight. The symbols O, Ω, Θ describe bound types, not worst, best, and average cases. Figure 3 illustrates the comparison beyond n_0 .

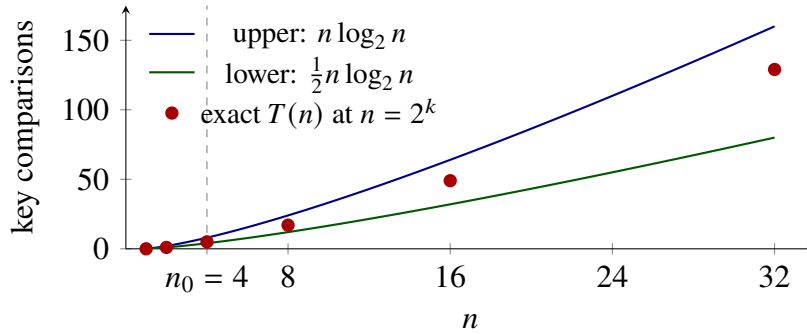


Figure 3: Bounding functions and exact merge-sort counts at powers of two. Both bounds hold at these input sizes for $n \geq n_0 = 4$ (Section 6).

6 Merge Sort and Its Recurrence

Splitting and merging. Sorting returns the original entries in non-decreasing order, preserving multiplicity. Merge sort returns lists of length at most one immediately. For $n \geq 2$, it recursively sorts $\text{left} := A[:\lfloor n/2 \rfloor]$ and $\text{right} := A[\lfloor n/2 \rfloor:]$, then merges their sorted outputs. The floor $\lfloor n/2 \rfloor$ is the greatest integer no larger than $n/2$. During merging, compare the first remaining value in each list and append the smaller one to the output. Once a list is exhausted, append the other remainder (Knuth, 1998, Section 5.2.4). Each appended value is the smallest remaining value, so merging preserves sorted order and every occurrence. Recursive lengths decrease to the base case, which establishes termination and, recursively, correctness.

Listing 1: Merge sort returning a sorted copy. Only $\text{left}[i] \leq \text{right}[j]$ is counted as a key comparison.

```

1 def merge_sort(A):
2     if len(A) <= 1:
3         return list(A)
4     mid = len(A) // 2
5     return merge(merge_sort(A[:mid]), merge_sort(A[mid:]))
6
7 def merge(left, right):
8     out, i, j = [], 0, 0
9     while i < len(left) and j < len(right):
10        if left[i] <= right[j]:
11            out.append(left[i]); i += 1
12        else:
13            out.append(right[j]); j += 1
14    out.extend(left[i:])
15    out.extend(right[j:])
16    return out

```

The cost of a merge. For sorted lists of lengths $a, b \geq 1$, merging uses at most $a + b - 1$ comparisons. Each comparison consumes one value, and at least one value remains to be appended without another comparison. For example, merging $\langle 1, 3, 5 \rangle$ and $\langle 2, 4, 6 \rangle$ requires the five comparisons 1:2, 3:2, 3:4, 5:4, 5:6. Here the colon separates the two compared values. The final 6 is appended directly.

For $n = 2^k$, where k is a non-negative integer, each split has equal halves. The merge bound gives $T(n) \leq 2T(n/2) + n - 1$. Equality is attainable: place odd-ranked values in one half and even-ranked values in the other, with a worst-case arrangement recursively within each half. Relabelling in order preserves each half's comparisons, and the sorted halves interleave until the last comparison. Therefore

$$T(n) = 2T(n/2) + n - 1 \quad (n \geq 2), \quad T(1) = 0. \quad (1)$$

These costs count comparisons only. Figure 4 shows the subproblem sizes.

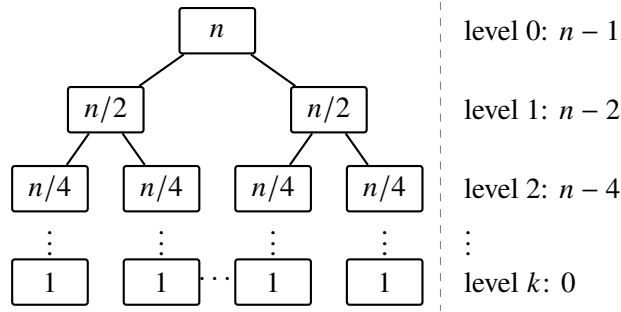


Figure 4: Subproblem sizes and total merge costs at each depth. The bottom row represents all n singleton leaves schematically.

Expanding the recurrence. Successive substitutions in Equation (1) give

$$\begin{aligned} T(n) &= 2T(n/2) + (n - 1) \\ &= 4T(n/4) + (n - 2) + (n - 1) \\ &= 8T(n/8) + (n - 4) + (n - 2) + (n - 1). \end{aligned}$$

After j substitutions, where $1 \leq j \leq k$, the corresponding identity is

$$T(n) = 2^j T(n/2^j) + \sum_{i=0}^{j-1} (n - 2^i) = 2^j T(n/2^j) + jn - (2^j - 1).$$

At depth $j < k$, there are 2^j subproblems, each of size $n/2^j$. Their total merge cost is $2^j(n/2^j - 1) = n - 2^j$. After $k = \log_2 n$ levels, the $2^k = n$ leaves each cost $T(1) = 0$. Consequently,

$$\begin{aligned} T(n) &= 2^k T(1) + \sum_{j=0}^{k-1} (n - 2^j) \\ &= kn - (2^k - 1) = n \log_2 n - n + 1 \leq n \log_2 n. \end{aligned} \tag{2}$$

The geometric sum is $\sum_{j=0}^{k-1} 2^j = 2^k - 1$ (Graham et al., 1994). Its last index is $k - 1$ because depth k contains only base cases. The number of nonzero cost levels is k , not 2^k . Thus the classroom conclusion is $T(n) = O(n \log n)$ on powers of two. For $n \geq 4$ in this domain, $\log_2 n \geq 2$ gives $\frac{1}{2}n \log_2 n \leq T(n) \leq n \log_2 n$. This proves the tighter statement $T(n) = \Theta(n \log n)$. Changing a fixed logarithm base only multiplies the bound by a constant.

Checking the exact solution. Induction verifies a base case and then shows that the result for smaller inputs implies the result at the next size. Equation (2) gives zero at $n = 1$, matching $T(1) = 0$. Assuming the formula at $n/2$ gives

$$\begin{aligned} 2T(n/2) + n - 1 &= 2 \left[\frac{n}{2} \log_2 \frac{n}{2} - \frac{n}{2} + 1 \right] + n - 1 \\ &= n(\log_2 n - 1) - n + 2 + n - 1 \\ &= n \log_2 n - n + 1. \end{aligned}$$

This proves the formula for every $n = 2^k$ by induction on k .

For other input lengths, the halves have sizes $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$, where the ceiling rounds upward. Rounding changes the exact count but preserves $\Theta(n \log n)$ growth: there are at most $\lceil \log_2 n \rceil$ merge levels with at most n comparisons per level. At each of the first $\lfloor \log_2 n \rfloor$ levels, the attainable worst-case cost is $n - 2^j$. Attainment follows by separating the two largest values at every split and recursively choosing worst-case orders in both halves. Summing those levels gives a matching lower bound (Cormen et al., 2001, Chapter 4).

7 Worked Example

Consider eight patient heights, in centimetres:

$$A = \langle 171, 158, 180, 152, 163, 149, 175, 160 \rangle.$$

Table 1 records the seven merges in execution order. Depth is measured from the root, with depth zero for the final merge.

Depth	Left	Right	Merged output	Count
2	$\langle 171 \rangle$	$\langle 158 \rangle$	$\langle 158, 171 \rangle$	1
2	$\langle 180 \rangle$	$\langle 152 \rangle$	$\langle 152, 180 \rangle$	1
1	$\langle 158, 171 \rangle$	$\langle 152, 180 \rangle$	$\langle 152, 158, 171, 180 \rangle$	3
2	$\langle 163 \rangle$	$\langle 149 \rangle$	$\langle 149, 163 \rangle$	1
2	$\langle 175 \rangle$	$\langle 160 \rangle$	$\langle 160, 175 \rangle$	1
1	$\langle 149, 163 \rangle$	$\langle 160, 175 \rangle$	$\langle 149, 160, 163, 175 \rangle$	3
0	$\langle 152, 158, 171, 180 \rangle$	$\langle 149, 160, 163, 175 \rangle$	$\langle 149, 152, \dots, 180 \rangle$	7
Total key comparisons				17

Table 1: The seven merges for the eight-patient example. Count refers only to comparisons between heights. The complete final output is given in the text.

The final merge compares the pairs 152:149, 152:160, 158:160, 171:160, 171:163, 171:175, and 180:175. The remaining 180 is appended directly. The output is $\langle 149, 152, 158, 160, 163, 171, 175, 180 \rangle$. The median is $(160 + 163)/2 = 161.5$ cm.

The total cost is $4 + 6 + 7 = 17 = 8 \log_2 8 - 8 + 1$, attaining $T(8)$. Each merge reaches its $a + b - 1$ bound. An already sorted input of eight distinct values instead uses $4 + 4 + 4 = 12$ comparisons with this implementation. An individual input's cost can therefore be lower than $T(n)$.

8 Summary

Membership differs from inclusion, and counting requires distinct labels. Quantifier order determines whether a choice may depend on an input. Functions assign one output per input, including costs indexed by list length. Asymptotic bounds compare growth using fixed constants and thresholds. For merge sort, summing $n - 2^j$ comparisons across recursion levels yields $n \log_2 n - n + 1$ when $n = 2^k$, and $\Theta(n \log n)$ growth overall.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2001). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 2nd edition.
- Graham, R. L., Knuth, D. E., and Patashnik, O. (1994). *Concrete Mathematics: A Foundation for Computer Science*. Addison-Wesley, Reading, MA, 2nd edition.
- Halmos, P. R. (1960). *Naive Set Theory*. D. Van Nostrand Company, Princeton, NJ.
- Knuth, D. E. (1976). Big omicron and big omega and big theta. *ACM SIGACT News*, 8(2):18–24.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2nd edition.